



## PROJECT DOCUMENTATION / PART 1 OF 2

# Routing, fuel calculations and optimisation

A mathematical and operational explanation of how Fuelio finds road routes, selects eligible fuel stops, calculates purchases, and proves the best plan within its model.

Prepared for Div  
13 September 2026  
Archive: Fuelio-main.zip

This manual describes the supplied implementation. It separates implemented behavior, verification evidence, known limitations, and suggested future work. No source files or deployed services were changed.

Archive SHA-256

59d07061988884f8ce301b76d32a2063d3cb560b264d0c303a0842c563bec7c3

# Contents

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1. What this volume establishes</b>                   | <b>5</b>  |
| 1.1 Source and verification boundary . . . . .           | 5         |
| 1.2 The precise meaning of cheapest . . . . .            | 5         |
| 1.3 The end-to-end pipeline . . . . .                    | 6         |
| <b>2. Inputs, units and invariants</b>                   | <b>7</b>  |
| 2.1 Units and coordinate conventions . . . . .           | 7         |
| 2.2 Rules and defaults . . . . .                         | 7         |
| 2.3 Destination fuel . . . . .                           | 8         |
| 2.4 Full and Half Tank are departure targets . . . . .   | 8         |
| <b>3. Route finding and address resolution</b>           | <b>9</b>  |
| 3.1 Live address suggestions . . . . .                   | 9         |
| 3.2 Ordered waypoints . . . . .                          | 9         |
| 3.3 Ordinary road routes: OSRM . . . . .                 | 9         |
| 3.4 Optional truck routes: openrouteservice . . . . .    | 10        |
| 3.5 What requires another Find routes . . . . .          | 10        |
| <b>4. Turning a road geometry into eligible stops</b>    | <b>11</b> |
| 4.1 Route preparation . . . . .                          | 11        |
| 4.2 Projection onto the whole segment . . . . .          | 11        |
| 4.3 Repeated passes and same-mile alternatives . . . . . | 11        |
| 4.4 One mile means geographic corridor first . . . . .   | 11        |
| <b>5. Directed road access and detour economics</b>      | <b>13</b> |
| 5.1 The return-to-route model . . . . .                  | 13        |
| 5.2 Ordinary access matrices . . . . .                   | 13        |
| 5.3 Truck access and connector geometry . . . . .        | 13        |
| 5.4 Caches, pacing and completeness . . . . .            | 13        |
| <b>6. Fuel mathematics, step by step</b>                 | <b>15</b> |
| 6.1 Symbols . . . . .                                    | 15        |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| 6.2 First purchase . . . . .                                           | 15        |
| 6.3 Later purchases . . . . .                                          | 15        |
| 6.4 Finish without another stop . . . . .                              | 15        |
| 6.5 Whole-trip identities . . . . .                                    | 16        |
| 6.6 Numerical tolerances . . . . .                                     | 16        |
| <b>7. The exact dynamic programme</b>                                  | <b>17</b> |
| 7.1 Why a small state is sufficient . . . . .                          | 17        |
| 7.2 State, layers and initial condition . . . . .                      | 17        |
| 7.3 Relaxation and dominance . . . . .                                 | 17        |
| 7.4 Reconstruction . . . . .                                           | 17        |
| 7.5 Proof outline . . . . .                                            | 18        |
| 7.6 Complexity and scaling . . . . .                                   | 18        |
| <b>8. Buffer, strict window and California modes</b>                   | <b>19</b> |
| 8.1 Plain Buffer . . . . .                                             | 19        |
| 8.2 Strict arrival window . . . . .                                    | 19        |
| 8.3 Rescue behavior . . . . .                                          | 19        |
| 8.4 Before California . . . . .                                        | 19        |
| 8.5 Boundary geometry and precision . . . . .                          | 19        |
| <b>9. Worked calculations</b>                                          | <b>21</b> |
| 9.1 Half Tank: arriving with 119 gallons . . . . .                     | 21        |
| 9.2 A directional detour . . . . .                                     | 21        |
| 9.3 Lowest station price need not mean lowest purchase spend . . . . . | 21        |
| 9.4 A strict-window surprise . . . . .                                 | 22        |
| 9.5 No purchase is often the best answer . . . . .                     | 22        |
| <b>10. Results, replay and failure handling</b>                        | <b>23</b> |
| 10.1 Successful output contract . . . . .                              | 23        |
| 10.2 Arithmetic replay . . . . .                                       | 23        |
| 10.3 Infeasibility versus error . . . . .                              | 23        |
| 10.4 Cancellation and partial alternatives . . . . .                   | 24        |
| 10.5 Messages do not change a solved plan . . . . .                    | 24        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>11. Retained route library and historical-load calculation</b> | <b>25</b> |
| 11.1 Visibility versus execution . . . . .                        | 25        |
| 11.2 The 12 bundled directional lanes . . . . .                   | 25        |
| 11.3 Load parsing and overrides . . . . .                         | 25        |
| 11.4 City clustering . . . . .                                    | 25        |
| 11.5 Directional lane eligibility . . . . .                       | 26        |
| <b>12. Verification evidence and limits of the model</b>          | <b>27</b> |
| 12.1 What was verified . . . . .                                  | 27        |
| 12.2 What remains outside the guarantee . . . . .                 | 28        |
| 12.3 Safe extension boundaries . . . . .                          | 28        |
| 12.4 Diagnostic order for a surprising plan . . . . .             | 28        |
| <b>A. Planning source index</b>                                   | <b>30</b> |

# 1. What this volume establishes

Fuelio is a US-focused dispatch workspace that combines live vehicle inputs with a fuel-purchase planner. Its route finder obtains road geometries from a routing provider. Its separate optimiser chooses fuel stops along each supplied geometry. Those are distinct calculations with different guarantees.

This volume describes the exact source in the uploaded Fuelio-main.zip. It covers all calculation modules, the browser/server routing integration, retained historical-load workflows, output arithmetic, verification evidence and known mathematical limits. Part II documents the rest of the application, including the detailed Samsara and Google Sheets connections that supply these inputs.

## 1.1 Source and verification boundary

The archive contains 34 files. No previous screenshot, earlier requirement or README sentence overrides the executable behavior of this snapshot. The review included every source file, both manifests, the dependency lock, templates, route definitions, the complete California geometry and visual assets. File hashes and the full inventory appear in Part II.

The archive was copied to a separate verification directory. Its build completed, and all 53 tests passed under Node v24.19.0. The package declares Node 22.x for deployment; this documentation run did not execute the tests under Node 22. Tests include 600 generated optimiser comparisons against an independent exhaustive enumerator. Provider responses and browser map objects in integration tests are simulated. No production fleet, private Google workbook or live route was queried during verification.

## 1.2 The precise meaning of cheapest

The objective is the smallest sum of dollars spent buying fuel during the trip, subject to the entered rules. Fuel already in the vehicle is treated as owned. Fuel left at the destination is reported but has no resale credit in the objective. There are no separate penalties for time, tolls, stops, wages, idling or inconvenience. Road-access miles consume fuel, so they can affect the amount bought and its cost.

An optimal result is an exact minimum for the supplied route, accepted stop occurrences, directional access distances, prices, constant MPG and fixed full/half-fill rules. It is not a proof of the cheapest possible journey across every road, every station or arbitrary purchase quantities.

The browser compares successful plans for the selected returned routes. A badge such as shortest or fastest describes the alternatives that the provider returned, not all possible paths in the road network. A completed result is also a snapshot of its source prices and form values; it is not continuously re-optimised as the fleet moves.

### 1.3 The end-to-end pipeline

| Stage               | Input and operation                                                                 | Output                                                              |
|---------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Trip input          | Selected truck or manual origin, destination, ordered waypoints and routing profile | Ordered geographic points                                           |
| Road route          | OSRM ordinary-road request or optional ORS HGV request                              | One or more polylines, route miles, provider driving duration       |
| Route preparation   | Convert geometry, measure every segment and scale progress to provider distance     | Route positions with cumulative mileposts                           |
| Corridor search     | Project every loaded stop onto all relevant segments                                | Eligible stop occurrences within the geographic radius              |
| Access confirmation | Compute anchor-to-stop and stop-to-anchor road distances                            | Directional spur mileage or confirmed exclusion                     |
| Optimisation        | Enumerate all feasible forward transitions with dynamic programming                 | Lowest-purchase-cost fixed-fill stop sequence                       |
| Replay and output   | Recompute fuel balance and costs; attach source/trip metadata                       | Auditable result cards, chart, CSV, JSON and editable message draft |

Source: optimizer.js; geo.js; workspace.js:1038-1587; server.js:694-998.

## 2. Inputs, units and invariants

### 2.1 Units and coordinate conventions

| Quantity                       | Internal convention           | Consequence                                                   |
|--------------------------------|-------------------------------|---------------------------------------------------------------|
| Distance                       | Statute miles                 | Provider metres are divided by 1,609.344.                     |
| Fuel                           | US gallons                    | Report millilitres are divided by 3,785.411784.               |
| Efficiency                     | Miles per US gallon           | Every road leg consumes leg miles divided by MPG.             |
| Fuel price                     | Dollars per US gallon         | Positive numeric values only; no exchange-rate conversion.    |
| Duration                       | Seconds from routing provider | Displayed in hours/minutes; not used as an optimisation cost. |
| User coordinates               | Latitude, longitude           | Manual coordinate text is parsed in that order.               |
| GeoJSON / provider coordinates | Longitude, latitude           | The server serialises this order explicitly.                  |
| Truck dimensions               | Metres; weight in tonnes      | Used only for optional HGV route restrictions.                |

The data parser can read a number with a dollar sign, percent sign or conventional thousands separators. It removes those display characters; it does not infer currencies or convert percentages to fractions. An entered MPG is a constant trip assumption. Road grade, load weight, current speed, wind, weather and idle fuel are not used to vary consumption.

### 2.2 Rules and defaults

The fresh HTML form starts with the defaults below. Browser persistence can restore previous values, and vehicle selection can replace fuel and MPG with telemetry or a manual override. Therefore the screen, not this default table, determines the next calculation.

| Rule                  | Fresh UI default | Solver validation / meaning                                                             |
|-----------------------|------------------|-----------------------------------------------------------------------------------------|
| Tank capacity C       | 240 gal          | Numeric 0.01 to 5,000.                                                                  |
| Starting fuel F       | 100 gal          | Between zero and C.                                                                     |
| Starting fuel percent | 41.67%           | Form convenience; converted using the entered capacity. Not a separate solver variable. |
| MPG m                 | 7                | Numeric 0.01 to 100.                                                                    |
| Filling strategy      | Full Tank        | Exactly Full or Half; target $Q = C$ or $C/2$ .                                         |
| Buffer / reserve B    | 50 gal           | Between zero and C. Its effective meaning depends on mode.                              |

| Rule                  | Fresh UI default | Solver validation / meaning                                              |
|-----------------------|------------------|--------------------------------------------------------------------------|
| Destination target E  | 50 gal           | Minimum fuel, not exact fuel unless tolerance is zero.                   |
| Maximum stops K       | 3                | Whole number from zero to 1,000; runtime caps layers at candidate count. |
| Route corridor        | 1 mile           | Geographic distance from a route segment; part of candidate preparation. |
| Mode                  | Plain Buffer     | buffer, strict, or before_ca.                                            |
| Lower / upper offsets | 10 / 5 gal       | Each between zero and C; used for the strict arrival window.             |
| Ending tolerance      | Blank            | Blank means no upper destination target below tank capacity.             |
| Rescue                | Off              | Optional retry for an infeasible strict arrival-window plan.             |

The solver receives numbers, not form strings. Its numeric validator rejects NaN, infinity, missing required values and strings that merely look numeric. The UI converts form text with the shared parser before calling the solver. The corridor and routing dimensions have their own UI/server validation; they are not optimiser-state variables.

## 2.3 Destination fuel

Let  $H$  be the effective hard reserve. The required minimum at the destination is  $\max(E, H)$ . With no ending tolerance, the upper limit is  $C$ . With tolerance  $T$ , the upper limit is  $\min(C, E + T)$ . Both limits allow a small numerical epsilon.

This distinction matters:  $E = 50$  and blank tolerance permits arriving with 140 gallons. It does not ask the solver to spend fuel until only 50 remains.  $E = 50$  and  $T = 0$  requires approximately 50. An upper limit below the effective reserve makes the plan infeasible. The optimiser does not dispose of excess fuel to satisfy a target.

## 2.4 Full and Half Tank are departure targets

At every selected refuelling stop, the truck leaves with  $Q$  gallons. Under Half Tank with  $C = 240$ ,  $Q = 120$ . Arrival with 119 therefore causes a purchase of 1 gallon. Arrival with 80 causes a purchase of 40. Arrival with 130 cannot be a half-fill purchase because that would require removing fuel; the truck must continue or finish without that stop.

Starting fuel may exceed the chosen target. That is valid: a vehicle with 196 gallons can use Half Tank mode, consume its existing fuel and make a later positive purchase to 120. A zero-gallon transaction is not counted as a stop and cannot be used to satisfy the California pre-entry-stop requirement. If Half Tank is infeasible, the result offers a Full Tank action; there is no silent full-tank substitution.

Source: index.html planner inputs; workspace.js:297-348; optimizer.js:12-65, 219-255.

## 3. Route finding and address resolution

### 3.1 Live address suggestions

Origin, destination and each separate waypoint use the same place-search controller. After 450 ms without another input event, a query of at least three characters asks `/api/suggest`. A sequence number, `AbortController`, unchanged-query check and DOM-connected check reject obsolete responses. Clicking outside or pressing Escape closes the list. Typing again starts a new request. Arrow keys select options; Enter accepts a highlighted suggestion or makes an explicit search.

The suggestion service is Photon, configured by `PHOTON_URL` or the default `photon.komoot.io` endpoint. The server requests English US results, up to six, and filters returned country codes and coordinates. Its queue spaces requests by 600 ms per warm server instance. Suggestions have a 12-second total deadline and a one-day cache. Direct coordinate text bypasses external geocoding.

Explicit Enter/search uses `/api/geocode` and a Nominatim-compatible service. It accepts query lengths of 2-250 characters, requests US results with address details and limit six, and caches by lower-cased query for one day. Requests are paced at 1,100 ms per server instance. Nominatim is not used for live autocomplete. This separation is intentional; the public Nominatim policy prohibits autocomplete and imposes an application-wide usage limit, which an instance-local queue alone does not enforce across multiple instances.

If Find routes resolves a typed address without a selected suggestion, the helper takes the first returned geocoding result. Dispatchers should select a specific suggestion where an address is ambiguous. Display labels and coordinates are remembered together, and changing the label invalidates its previous selected coordinates.

Reference: [Nominatim usage policy](#); [Photon project](#). The request behavior above is from `place-search.js` and `server.js:694-764`.

### 3.2 Ordered waypoints

Waypoints are separate editable rows. Users can add, remove and reorder them. Each row accepts address text or latitude/longitude and has the same suggestion behavior as the endpoints. A hidden newline-separated field preserves compatibility with the older planning functions and persistence format. The route respects the entered order; it does not solve a travelling-salesperson problem or automatically reorder deliveries.

The routing API permits 2-25 points, including origin and destination. Thus at most 23 intermediate points can be submitted in one request. Valid points must fall within a broad US-region rectangle: latitude 18 through 72 and longitude -180 through -60. This is a rectangular validation rule, not an exact US border test. Imported stop/load coordinates can be globally valid yet fail this routing-region check.

### 3.3 Ordinary road routes: OSRM

`POST /api/routes` receives points, profile and options. Any profile other than `hgv` follows the ordinary-road branch. Coordinates are serialised as longitude,latitude with six decimals. The server requests `route/v1/driving`, full overview, GeoJSON geometry and steps. With only two points, it requests `alternatives=3`; with waypoints, `alternatives=false`. The code returns all routes provided by the service. It does not promise exactly three alternatives or impose a three-route UI cap.

OSRM's Route service seeks fastest routes through the supplied points. In its API, a numeric `alternatives` parameter requests up to that many alternatives; alternatives are not guaranteed. Fuelio then labels the shortest/fastest among the returned options. It does not request OSRM's Trip service, map-matching service or a global fuel-aware route search.

The response is normalised to id, name, summary, distance in miles, duration in seconds, coordinates, profile and provider. Road summaries combine nonempty leg summaries; the fallback is Road route. Routes are

cached for six hours using the ordered points, profile and options. The browser assigns its own fresh IDs when preparing the route cards.

Standard routing does not check the truck's height, weight, length, width, hazardous cargo, toll preference or commercial-vehicle legality. Provider duration excludes Fuelio's refuelling dwell times and driver breaks; Fuelio does not add live traffic. A highway line shown on the map does not certify that a truck may legally use it.

Reference: [OSRM HTTP API](#). Source: `server.js:765-887`.

### 3.4 Optional truck routes: openrouteservice

ORS\_API\_KEY enables the HGV option. Fuelio sends an authenticated POST to `/v2/directions/driving-hgv/geojson` with ordered coordinates, `instructions=true` and an options object. Height and width must be positive and at most 10 metres; length at most 60 metres; weight at most 150 tonnes. Hazmat sets a restriction boolean. Avoid tolls sends `avoid_features` containing tollways. Fresh UI dimensions are 4.1 m high, 2.6 m wide, 22 m long and 36 tonnes.

Each returned feature becomes a route. Its summary is the first five distinct nonempty instruction road names. There is no request for `alternative_routes` in this branch. A single truck route is therefore normal. Waypoints can constrain a different corridor, but the hidden saved-lane page is not directly available to ordinary users.

The source labels the result Truck restrictions applied after sending these fields. That text is not an independently verified road-legality certificate. The current request does not explicitly set `vehicle_type`. Published ORS routing-options documentation discusses `vehicle_type` in connection with HGV restrictions. Provider behavior for this exact payload should be confirmed against the deployed ORS version before treating every dimensional restriction as proven effective. This documentation did not make a live ORS call.

Reference: [ORS routing options](#). Source: `server.js:785-858`.

### 3.5 What requires another Find routes

Fuelio records a trip stamp containing client, truck field, origin, destination, waypoint text, routing profile and truck-routing options. Changes to any of these invalidate the stamp. Fuelio Hunt requires a fresh route before solving. This also prevents a response from an older in-flight route request from replacing a newer trip.

Capacity, starting gallons, fuel percent, MPG, filling strategy, reserve, ending target, maximum stops, corridor and arrival-window rules do not require another directions request. Fuelio Hunt reads their current values and repeats candidate selection/access checking as necessary. Changing a corridor can change the candidate list while keeping the road geometry.

Live location refresh can change an unprotected origin field and therefore make an existing route stale. The app does not automatically reroute every minute. Manual overrides protect edited fields until Use live clears the override for that vehicle. Map display changes, themes and Price Heat do not affect a trip stamp or any calculation.

Source: `workspace.js:319-368, 1038-1198, 1358-1415`.

## 4. Turning a road geometry into eligible stops

### 4.1 Route preparation

`prepareRoute` converts provider [longitude, latitude] pairs to point objects and requires at least two points. It computes a cumulative great-circle length along every consecutive segment. The provider's positive total distance is preferred; otherwise the geometric sum is used. Cumulative distances are scaled by provider distance divided by geometric sum.

Scaling makes the end mile agree with the route provider while retaining proportional positions along the polyline. It does not establish a surveyed highway milepost. Polyline simplification, coordinate snapping and unequal segment accuracy can distort an intermediate stop's route position. Result Route mile is progress from the entered origin, not a state's signed exit mile number.

The geometric radius uses an Earth radius of 3,958.7613 miles. Haversine distance measures surface separation between two coordinates. Bearing and great-circle movement support projection, interpolation and polygon-entry refinement. Degenerate nearly zero-length segments receive a safe endpoint treatment.

### 4.2 Projection onto the whole segment

`routeCandidates` scans every stop against every relevant route segment. It first uses a latitude bounding check as a speed-up, then computes the closest clamped great-circle projection. It does not look only at polyline vertices. A stop halfway along a long segment can therefore qualify even if it is far from both endpoints.

For a segment from a to b and a stop p, the helper calculates bearings from a, an along-track angle, and a fraction clamped between zero and one. Moving that fraction of the segment yields an anchor on the selected route. Haversine distance from the anchor to p is the corridor offset. A candidate qualifies if offset is at most the configured radius plus a small epsilon.

Each candidate includes the original stop fields, an occurrence ID, its original physical pumpId, its computed route mile, offset and projection. Later stages add inbound and outbound road mileage. An imported route-mile column is not trusted in place of geometric projection.

### 4.3 Repeated passes and same-mile alternatives

A looping route may pass the same physical station more than once. The scanner groups a contiguous run of nearby segment hits, retaining the closest projection in that run. It can create another occurrence when there is a nonadjacent encounter separated by more than twice the corridor radius plus 0.5 route miles. Occurrences use visit-specific IDs, so the optimiser can consider a later pass independently.

This is a deliberate geometric heuristic for distinguishing visits; it is not an exact road-network station-entrance traversal detector. At an identical route mile, different stations are alternatives. The solver prevents a second purchase at the same or earlier progress after another stop, avoiding artificial zero-progress refuelling loops.

### 4.4 One mile means geographic corridor first

The current 1-mile eligibility setting is measured from any selected-route segment to the stop geographically. It is not a 1-mile road-distance limit. The later access calculation measures road miles, but the current code does not reject a candidate simply because its road detour exceeds one mile.

Consequently a station across a divided highway, river or access barrier may qualify geographically while requiring a long road approach. Confirmed access and its fuel are included in the plan, but there is no fixed

maximum actual access distance. There is also no highway-name equality rule, mandatory exit match, same-side rule or automatic widening when no stop is available.

Stop Highway and Exit columns supply display, filtering and messages. They do not drive the optimiser's eligibility check. Fuel Stops page search/state/highway filters are viewer controls; they do not silently reduce the planner's loaded source dataset. The planner uses the selected fuel source and its own corridor.

Source: geo.js:5-143; workspace.js:1204-1281; server.js:888-965.

## 5. Directed road access and detour economics

### 5.1 The return-to-route model

For a stop  $j$ ,  $a_j$  is road distance from its projected route anchor to the stop and  $b_j$  is road distance from the stop back to that same anchor. The two directions may differ because of one-way roads and interchange geometry. Fuelio must not replace either distance with a geographic offset.

If stop  $i$  was the previous purchase, the next leg is its outbound spur, the forward selected-route distance between anchors, then stop  $j$ 's inbound spur. The final leg includes the last stop's outbound spur and the remaining route to the destination. Every purchase therefore returns to the same anchor before proceeding. This makes the route order and fuel accounting consistent, while remaining an approximation to a fully rerouted journey through all selected station entrances.

An alternative route that leaves one highway and rejoins farther along it is not modelled as such. The approach/return provider can also snap an anchor in a way that does not enforce the selected route's travel direction. The ordinary request contains no bearing, curb-side or approach constraint. These are limits on the travel model, independent of the optimiser's mathematical exactness.

### 5.2 Ordinary access matrices

The browser sends missing candidate access checks in batches of up to 20. Each item contains an ID, projection and stop coordinates. The server validates the coordinates, caps the geographic anchor-to-stop separation at 10.1 miles, and truncates the transmitted ID to 250 characters. That 10.1-mile guard protects the API; it is not the user's one-mile corridor rule.

For ordinary routing, points are flattened as anchor0, stop0, anchor1, stop1 and so on. OSRM Table is asked for distances and durations. Cell  $2i$  to  $2i+1$  supplies inbound mileage; the reverse cell supplies outbound mileage. Those are the distances of the provider's fastest road paths, not necessarily the shortest-distance paths. Fuelio uses their distances and retains the combined duration for data/reference purposes; seconds do not enter purchase cost.

Null or missing directional distances mark that candidate unreachable. A stop-side snap more than 250 metres from its submitted point also marks it unreachable. A missing snap-distance field defaults to zero, so that specific safeguard depends on the provider returning it. A request-level failure is an error, not a permission to estimate straight-line access or label a partially checked route optimal.

### 5.3 Truck access and connector geometry

For HGV routing, the browser uses smaller batches of three. The server routes anchor to stop to anchor with the same ORS options and requires two returned segments. It processes these within the road-service pacing mechanism. Failure to obtain the expected segments rejects the request instead of fabricating a distance.

Selecting a recommended stop can request `/api/stop-path` to draw the approach and return geometry. This dashed connector is explanatory. It does not alter the already computed gallons, prices or chosen sequence. The result is still based on the directional distances used during planning. OSRM/ORS are not called by the worker itself.

### 5.4 Caches, pacing and completeness

Access entries are cached server-side for one day. The browser also stores entries for one day in IndexedDB, keyed by routing profile, routing options and rounded anchor/stop coordinates. The browser cache retains at most 3,000 entries. Confirmed unreachable outcomes are cached too; a previously unreachable stop can remain excluded until that cache expires or changes.

Price changes do not require recomputing road access because prices are not part of the geometric access key. The current price record is joined to cached access data before solving. A source dataset change cancels an active foreground solve and clears its displayed plans; background price refresh preserves already produced plan snapshots.

Ordinary road calls are spaced by 1,050 ms per warm server instance, or 100 ms when `OSRM_URL` is explicitly configured. The shared road queue also wraps ORS calls. This pacing is process-local, not a distributed global provider quota. A large corridor or many alternative routes can make access preparation the slowest stage even though the worker calculation is exact and local.

Source: `workspace.js:1204-1281`; `server.js:44-111`, 763-998; OSRM API reference in Chapter 3.

## 6. Fuel mathematics, step by step

### 6.1 Symbols

| Symbol     | Meaning                                         |
|------------|-------------------------------------------------|
| D          | Selected route's base distance in miles.        |
| C, Q       | Tank capacity and fixed departure target.       |
| F, m       | Starting fuel and constant MPG.                 |
| B, H       | Entered buffer and effective hard reserve.      |
| E, T       | Destination fuel target and optional tolerance. |
| K          | Maximum number of positive fuel purchases.      |
| $x_j$      | Route progress of stop occurrence j.            |
| $a_j, b_j$ | Inbound and outbound road-access miles for j.   |
| $p_j$      | Price per gallon at j.                          |
| $A_j, G_j$ | Arrival gallons and purchased gallons at j.     |

### 6.2 First purchase

The first leg travels  $x_j$  route miles plus  $a_j$  inbound miles. Arrival gallons are  $F - (x_j + a_j)/m$ . If that arrival meets the mode's window and the purchase is positive,  $G_j = Q - A_j$ . Stop cost is  $p_j$  times  $G_j$ . The truck then leaves the station with Q, not Q plus arrival fuel.

The solver separately checks that returning to the route does not violate the reserve:  $Q - b_j/m$  must be at least H. This prevents a station from appearing feasible merely because arrival at the forecourt was feasible.

### 6.3 Later purchases

When i precedes j, leg distance is  $b_j + (x_j - x_i) + a_j$ . Since departure from i is always Q, arrival at j is Q minus this leg distance divided by m. Buying to Q replaces the gallons consumed on that leg. Purchases and costs use unrounded floating-point quantities; result formatting rounds only the display.

The purchase must be greater than EPS and must not exceed capacity minus arrival. Forward progress must be strictly greater than the previous stop's progress plus EPS. There are no negative purchases, disposal decisions or artificial visits that buy zero.

### 6.4 Finish without another stop

From the origin, destination fuel is  $F - D/m$ . From the last stop i, it is  $Q - ((D - x_i) + b_i)/m$ . If this is between the effective destination minimum and maximum, the state can finish. A zero-stop result is a valid optimum when starting fuel is sufficient. Before California mode is the exception because it also requires a qualifying pre-entry purchase.

## 6.5 Whole-trip identities

```
Actual miles = D + sum(inbound miles + outbound miles)
Fuel burned = Actual miles / MPG
Purchased gallons = sum(stop purchase gallons)
Ending fuel = Starting fuel + Purchased gallons - Fuel burned
Purchase cost = sum(stop purchase gallons * stop price)
```

The route-mile column and trip-mile column differ. Route mile is a station anchor's position on the base route. tripMile is actual distance driven when arriving at the station, including preceding spurs and the current inbound spur. The chart uses actual trip distance. The final result includes routeMiles, totalMiles, detourMiles, burned and finalLegMiles so the accounting can be reconstructed.

No intermediate gallon rounding, fixed fuel grid or integer-gallon purchase rule appears in the solver. If operations require a pump to dispense rounded gallons, the entered plan is an approximation to that transaction and should be recalculated/reviewed; the code does not currently solve a rounded-dispensing model.

## 6.6 Numerical tolerances

EPS is 0.0000001 for feasibility and cost comparisons. The replay uses a looser 0.00001 tolerance for recomputed quantities. Import duplicate prices have a 0.00000001 comparison threshold. Duplicate station keys use route miles rounded to seven decimals. Those are numerical safeguards, not operational fuel reserves.

Costs that differ by less than EPS are treated as ties. With ordinary US fuel prices, the difference is far below a cent, but it means the implementation is numerically exact to its tolerance rather than a symbolic rational solver. User-visible prices have four decimal places; money normally displays two; purchase and fuel quantities display two.

Source: optimizer.js:1-309; workspace.js:1416-1587.

## 7. The exact dynamic programme

### 7.1 Why a small state is sufficient

If purchase quantities were free continuous decisions, two visits to the same station could leave with different fuel, creating different future possibilities. Fuelio's full/half target removes that ambiguity. After every positive purchase, departure fuel is  $Q$ . Future feasibility therefore depends on the last stop, how many stops have been used, and whether the required California pre-entry purchase has occurred. Earlier history affects accumulated cost, but not future departure fuel.

The optimiser implements a resource-constrained shortest path on a forward-only station graph. The resource bound is the allowed number of stops. Each candidate is a potential purchase node, with origin and destination handled as initial/final states. There is no solver license, external linear-programming package or network request in this stage.

### 7.2 State, layers and initial condition

The code creates layers 0 through  $\min(K, N)$ , where  $N$  is the candidate count. Each layer is a Map keyed by the last candidate index and California flag. A stored state includes index, cost, prev pointer, ca flag, fuel, stops and actual miles so far. The initial state is index -1, zero cost, zero stops,  $F$  gallons and zero driven miles.

Candidates are validated, duplicate occurrences are reconciled, and the list is sorted by route mile then ID. Every state first checks whether it can finish at the destination. If more purchases are permitted, it examines every later candidate, not just the cheapest few or nearest few. Arrival window, positive purchase, capacity, outbound reserve and California conditions decide whether an edge is feasible.

### 7.3 Relaxation and dominance

For a feasible transition to  $j$ , new cost equals the previous state's cost plus  $p_j$  multiplied by  $Q - A_j$ . The new state records  $Q$  gallons, an incremented stop count, updated driven distance and a predecessor pointer. If the same layer/index/California flag already has a state, the cheaper one wins. Within cost tolerance, fewer actual miles wins.

That dominance is valid because both states arrive at the same purchase node with the same future departure fuel and the same remaining stop allowance. Every possible future suffix is equally feasible for both, with identical additional fuel purchases. Keeping the more expensive prefix cannot improve any final solution.

Destination candidates are compared by cost, then fewer stops, then fewer total miles. The browser's separate cross-route sorting is by total cost; it does not apply the full solver's stop-count and mileage tie-break across different route cards. Equal-cost cross-route display order follows the array's existing order.

### 7.4 Reconstruction

Once the best destination state is identified, predecessor pointers are followed back to the origin and reversed. Each chosen stop includes its source fields plus arrival, gallons, departure, cost, legMiles and tripMile. Totals are derived from that sequence. The output retains the validated rules and records whether rescue was used.

The worker receives one structured input message and posts either the solved result or an error message. The browser terminates the worker after completion or cancellation. There is no worker pool or persistent solver session. Moving the heavy search into a worker keeps the main browser thread available, although large input preparation and map rendering still occur outside the worker.

## 7.5 Proof outline

- Every permitted fuel purchase lies at a supplied candidate occurrence and has fixed departure  $Q$ .
- Every permitted sequence follows strictly increasing route progress and uses at most  $K$  purchases.
- The algorithm examines every forward transition from every retained feasible state, including finishing with no further purchase.
- States with the same future conditions can discard a more expensive prefix without losing a better completion.
- Induction across the stop-count layers therefore preserves a cheapest prefix for every reachable state.
- The minimum feasible destination cost is consequently the optimum for this finite fixed-fill model.

The proof assumes correct input distances, prices, candidate construction and constant consumption. It does not prove that the routing provider returned every useful route, that all truck entrances were represented, or that a driver's real fuel use will match the entered MPG.

## 7.6 Complexity and scaling

There are at most  $O(KN)$  retained station states and  $O(KN^2)$  forward transition checks. The extra Boolean state changes only a constant factor. Predecessor state storage is  $O(KN)$ . With a very large stop list, a large corridor, repeated passes or a high maximum stop count, runtime and memory grow materially. Setting  $K$  to 1,000 is accepted validation, not a promise of an interactive solve for arbitrarily many candidates.

There is no greedy price pruning, cheapest- $N$  cap, integer fuel grid or early shortcut that claims an approximate solution is exact. A future optimisation that prunes candidates or changes state dominance must preserve the proof or explicitly change the result's guarantee.

Source: optimizer.js:67-255; worker.js; workspace.js:1282-1310.

## 8. Buffer, strict window and California modes

### 8.1 Plain Buffer

H equals entered reserve B. Every considered arrival, outbound return and destination must preserve at least H. There is no upper arrival limit. A station can be selected while considerable fuel remains if buying to the target there creates the cheapest allowed sequence. The initial state must also have at least H; a truck starting below its entered buffer is immediately infeasible rather than being permitted an emergency first leg.

### 8.2 Strict arrival window

H is  $\max(0, B - \text{lower offset})$ . Refuelling arrival must be no greater than  $B + \text{upper offset}$ . For  $B = 50$ , lower = 10 and upper = 5, the accepted arrival interval is 40-55 gallons. The hard reserve throughout that strict-mode model is 40, not 50. This reflects the implemented lower edge of the window.

The destination minimum remains  $\max(E, H)$ , and fixed filling still applies. Increasing MPG can paradoxically remove a strict-window stop because the truck arrives with too much fuel. Increasing starting fuel can do the same. These are legitimate consequences of an upper arrival rule, not arithmetic faults.

### 8.3 Rescue behavior

Rescue is attempted only if strict mode has no feasible plan and the rescue option is enabled. It reruns the same complete search with the upper arrival limit removed. The lower reserve, target fill, ending limits, maximum stops and input stations remain unchanged. The result records `rescueUsed` and explains that the strict upper window was relaxed.

Rescue does not expand the corridor, invent a station, reduce the lower reserve, add stops beyond K, change Half to Full or alter the route. It is an optimum for the relaxed arrival-window model, not an optimum that satisfied the original strict upper bound. Plain Buffer and Before California do not invoke this strict-mode rescue retry.

### 8.4 Before California

The frontend requires a selected route that starts outside California and ends inside California. It locates the first entry mile using the bundled MultiPolygon. Physical stop coordinates inside that polygon are excluded before solving. The solver also rejects stations whose state is CA or whose progress is at/after the entry mile.

Effective hard reserve is 35 gallons. A purchase must arrive with 35-55 gallons, independent of the entered buffer and offset values. Every selected fuel purchase must occur before entry, and at least one qualifying pre-entry purchase is mandatory. After that, the truck must reach its destination and required ending fuel without purchasing inside California.

This is more specific than a general preference for cheap states. It does not minimise California gallons with an optional exception. It is not a full state-by-state fuel-tax model. A route ending elsewhere after passing through California fails the frontend mode gate, even though the geography helper can locate a crossing.

### 8.5 Boundary geometry and precision

`california.json` is a GeoJSON Feature with MultiPolygon geometry. Its properties identify California, state code 06, and a source note naming U.S. Census Bureau TIGERweb ACS2025 States layer 80, simplified to 0.0001 degrees. That source note is an assertion in the supplied file; the original boundary download was not repeated for this documentation.

The file contains 3,893 coordinate pairs, all finite. Its bounding rectangle is longitude -124.482003 to -114.131211 and latitude 32.529508 to 42.009503. The point-in-polygon routine handles rings and multipolygons. If the route starts inside, entry is null. Otherwise the first geometry vertex inside creates an outside/inside bracket, refined by 22 binary-search iterations along the route.

A segment that enters and exits California while both sampled endpoints remain outside can be missed. Boundary simplification and near-boundary floating-point behavior also limit precision. The result is a planning classification, not a legal boundary survey. These geometric limits do not disappear because the downstream fuel optimiser is exact.

Source: optimizer.js:19-65, 144-155, 219-255; geo.js:144-181; workspace.js:1311-1357; california.json.

## 9. Worked calculations

All examples below are synthetic arithmetic examples, not live station recommendations or current prices. The multi-stop example was run against the supplied optimiser during documentation.

### 9.1 Half Tank: arriving with 119 gallons

Set  $C = 240$ ,  $Q = 120$ ,  $m = 7$ ,  $F = 119$ ,  $H = 20$ ,  $E = 110$  and  $D = 70$ . Put one stop at mile zero with zero access miles and price \$4.00. Without buying, the truck finishes with 109 gallons and misses E. At that stop it buys exactly 1 gallon for \$4.00, leaves with 120, burns 10 gallons and arrives with 110. This shows why Half Tank means fill to 50% of capacity, not buy half a tank.

### 9.2 A directional detour

Set  $C = 240$ ,  $F = 100$ ,  $m = 7$ ,  $H = E = 50$  and  $D = 1,000$ . A stop at route mile 300 costs \$4.00/gal, has 3 inbound miles and 7 outbound miles.

| Calculation          | Result                                   |
|----------------------|------------------------------------------|
| First leg            | $300 + 3 = 303$ miles.                   |
| Arrival              | $100 - 303/7 = 56.7142857$ gal.          |
| Purchase to full     | $240 - 56.7142857 = 183.2857143$ gal.    |
| Purchase cost        | $183.2857143 \times 4 = \$733.1428571$ . |
| Return-to-route fuel | $240 - 7/7 = 239$ gal.                   |
| Final leg            | $1,000 - 300 + 7 = 707$ miles.           |
| Destination fuel     | $240 - 707/7 = 139$ gal.                 |
| Total miles          | $1,000 + 3 + 7 = 1,010$ .                |
| Fuel balance         | $100 + 183.2857143 - 1,010/7 = 139$ gal. |

Using only the inbound spur would understate the distance and overstate ending fuel. Treating both directions as three miles would also be incorrect. This is why access distances are directional.

### 9.3 Lowest station price need not mean lowest purchase spend

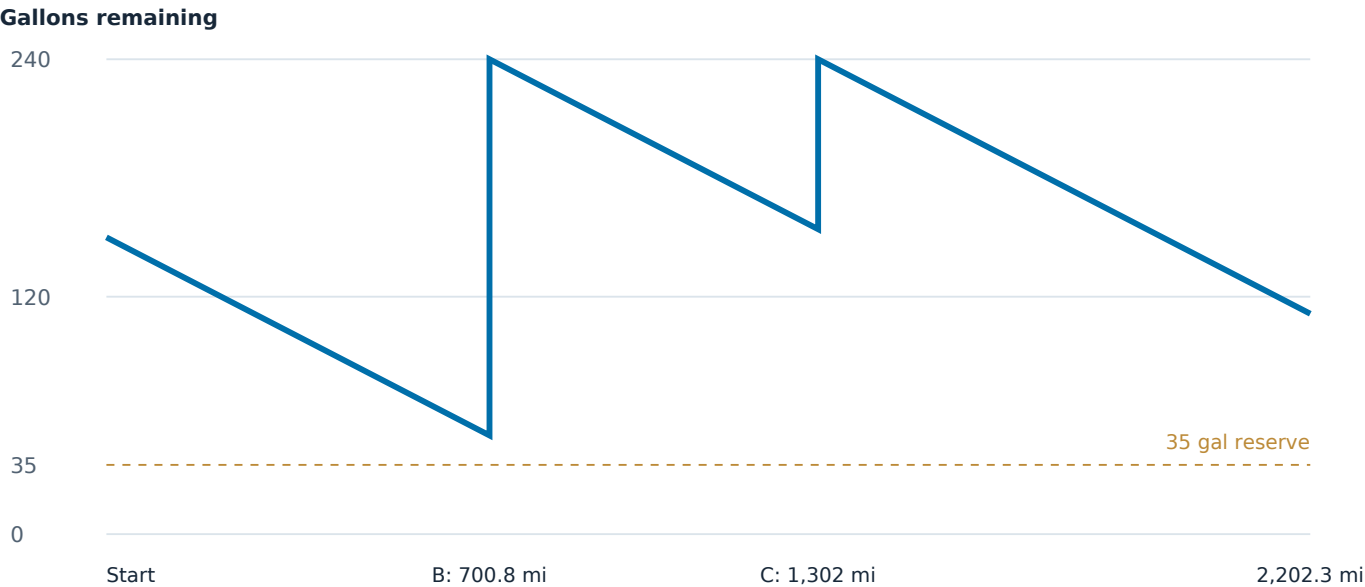
Set  $D = 2,200$ ,  $C = Q = 240$ ,  $F = 150$ ,  $m = 7$ ,  $H = 35$ ,  $E = 50$  and  $K = 3$ . Ending tolerance is blank. Available stops are:

| Stop | Route mile | Price / gal | In / out miles |
|------|------------|-------------|----------------|
| A    | 500        | \$4.25      | 0.5 / 1.0      |
| B    | 700        | \$3.75      | 0.8 / 0.9      |
| C    | 1,300      | \$4.10      | 0.3 / 0.3      |
| D    | 1,600      | \$3.50      | 1.0 / 1.0      |

The optimiser selects B then C. It buys 190.1142857 gallons at B for \$712.9285714, then 85.8857143 at C for \$352.1314286. Total purchase is 276 gallons and \$1,065.06. Driven distance is 2,202.3 miles, burn is

314.6142857 gallons and destination fuel is 111.3857143 gallons.

Replacing C with the lower-priced D would require refilling much later in the trip. After B, the B-to-D leg is 901.9 miles, requiring a purchase of 128.8428571 gallons at D. That sequence costs \$1,163.8785714 and carries approximately 154.1428571 gallons to the destination. The earlier, more expensive-per-gallon C wins on cash spent because the required full refill buys fewer gallons. No terminal fuel credit compensates the other plan for its extra remaining fuel.



The chart shows the chosen B/C plan. Vertical rises are purchases to 240. Falling segments use constant 7 MPG; the horizontal gold reference is the 35-gallon reserve. Distances on the axis include stop access.

With the same input but Half Tank, no feasible plan meets all constraints. The solver returns infeasible and suggests trying Full Tank. It does not mix full and half targets across stops.

9.4 A strict-window surprise

With C = 240, F = 100, m = 7 and a stop at mile 250 with zero access, arrival is 64.2857143 gallons. This is acceptable in Plain Buffer with reserve 50 but exceeds a strict 40-55 arrival window. If no other strict-window sequence works, rescue can accept the stop while retaining the 40-gallon lower reserve. The corresponding full purchase is 175.7142857 gallons. Rescue changes the constraint set; it does not change this fuel arithmetic.

9.5 No purchase is often the best answer

With F = 150, D = 500, m = 7 and E = H = 50, arrival without a stop is 78.5714286 gallons. Purchase cost is zero. A cheap station does not justify buying when the objective can be met without a purchase. A fuel dataset is not required for such a zero-stop plan, although road geometry still is.

Source: optimiser execution and formulas in optimizer.js. Tests include targeted Half Tank, zero-stop, spur, window and reserve cases.

# 10. Results, replay and failure handling

## 10.1 Successful output contract

| Field group      | Contents and meaning                                                                                 |
|------------------|------------------------------------------------------------------------------------------------------|
| Status           | optimal; rescueUsed and warnings qualify the applicable rule set.                                    |
| Money/fuel       | cost, gallons, endFuel, burned. These are raw numeric values before display rounding.                |
| Distance         | totalMiles, routeMiles, detourMiles and finalLegMiles.                                               |
| Sequence         | stops with original source metadata and arrival, gallons, departure, cost, legMiles, tripMile.       |
| Model            | Validated rules including target and hardReserve; candidateCount.                                    |
| Browser metadata | Route geometry, source name, createdAt, unique ID, truck/origin/destination and excluded-stop count. |

The result card shows total purchase cost, actual miles including access, gallons to buy, stop count, extra miles and destination gallons. Each row shows stop identity, location/highway/exit, route mile, arrive-buy-leave fuel and per-stop cost/price. The expandable fuel profile shows the complete fuel balance. CSV and JSON exports preserve more precision than the rendered screen.

The CSV has a 16-column stop table plus a TOTAL row. The TOTAL row's Route mile column contains total driven miles, while ordinary rows contain anchor progress; readers must distinguish those row types. JSON contains the complete plan object, including route and rules. A zero-stop CSV still includes a TOTAL row.

## 10.2 Arithmetic replay

verifyPlan reconstructs the route legs, recomputes arrival and departure fuel, positive purchases, cost, return-spur reserve, final fuel bounds and fuel mass balance. It checks maximum stops and the California rules. It runs before protected actions such as export, opening a plan message and reopening/restoring saved historical plans.

Replay is a consistency check on a produced plan, not a second optimiser. It does not call the routing provider, verify prices against a live workbook, prove global optimality or comprehensively validate arbitrary hostile JSON. Some redundant aggregate metadata are not independently compared, and it does not recreate the entire original candidate universe. Its strongest interpretation is that the model's recorded chosen sequence has a consistent fuel/cost calculation.

## 10.3 Infeasibility versus error

An infeasible result means the supplied valid model has no allowed sequence. Examples include starting below reserve, a first reachable purchase too far away, a large gap between stations, too few stops, half capacity below practical needs, an impossible ending window or no qualifying California stop. The response includes a reason and usually candidateCount; the immediate below-reserve return has a smaller result shape.

An error instead means that input validation, route geometry, source parsing, road access or worker execution could not establish a model/result. Invalid prices, conflicting duplicate occurrences, missing access responses and unavailable providers must not be treated as proof that no physical plan exists. The UI reports these failures and retains manual planning inputs.

## 10.4 Cancellation and partial alternatives

Selected routes are solved sequentially from captured rules and fuel data. The browser can abort its current request/controller and terminate the worker. Changing core trip/source state invalidates the active solve. In-flight server work is not universally cancelled merely because the browser disconnected; provider timeouts and server deadlines still matter.

If a later selected route fails, an earlier completed route may remain visible. A lowest-spend badge is therefore a comparison among successful results already available, not a proof that an uncompleted route was more expensive. Review the progress/error state before treating a partial route comparison as complete.

## 10.5 Messages do not change a solved plan

The current shared message editor copies recommended stops into editable primary fields and leaves alternative fields blank. Changes to those messages do not modify the optimiser's result. Messages intentionally omit calculated gallon instructions. Their default prose asks the driver to fill up the tank even for a Half Tank plan; that template language is not generated from the filling target. This is a current operational mismatch, documented rather than silently changed.

Source: optimizer.js:219-309; workspace.js:1358-1647; messages.js; message-editor.js.

# 11. Retained route library and historical-load calculation

## 11.1 Visibility versus execution

Routes & loads is hidden from navigation and page access. Its route-library, saved-route, import, clustering, batch and export code/data remain in the archive. The normal Find routes function does not automatically combine those predefined lanes with the entered trip. It requests the current origin, destination and visible ordered waypoints. Loading route-library.json at startup is not the same as using its routes in every calculation.

This distinction corrects an earlier conversation assumption that every library feature was always involved in ordinary planning. The hidden workflows are retained for later reactivation, not deleted; their algorithmic behavior is documented here so reactivation can be deliberate.

## 11.2 The 12 bundled directional lanes

| IDs             | Directional lane pair           | Stored special waypoint    |
|-----------------|---------------------------------|----------------------------|
| pur-1 / pur-2   | Fresno to Cheshire; reverse     | Tulsa in both directions.  |
| pur-3 / pur-4   | Fresno to Baltimore; reverse    | No intermediate waypoints. |
| pur-5 / pur-6   | Fresno to Chicago; reverse      | No intermediate waypoints. |
| pur-7 / pur-8   | Sacramento to Chicago; reverse  | No intermediate waypoints. |
| pur-9 / pur-10  | Los Angeles to Chicago; reverse | No intermediate waypoints. |
| pur-11 / pur-12 | Fresno to Atlanta; reverse      | No intermediate waypoints. |

The library stores coordinate anchors, names and waypoint text, not cached route polylines or guaranteed truck paths. For example, the entry labelled Baltimore stores 39.0458, -76.6413; this documentation preserves those values without certifying that they represent the desired facility or city centre. The source coordinates are enumerated in Part II's data reference.

## 11.3 Load parsing and overrides

Load rows require valid pickup and delivery latitude/longitude. Labels are optional. Accepted aliases cover pickup/origin and delivery/destination naming. The ID uses LoadID, TripID or ID, falling back to Load N. Optional StartingFuel, TankCapacity, MPG and EndingFuel numbers become per-load overrides. Invalid coordinate rows fail the import rather than silently becoming incomplete trips. Blank or unparseable optional numeric overrides are omitted, leaving current base rules in effect.

Every imported load begins with Imported status and an empty matches array. Preparing loads computes descriptive city groups and matches directional route definitions. The route used for a load always starts and ends at its actual coordinates, not at the library's city-centre anchors.

## 11.4 City clustering

clusterCities is equivalent to DBSCAN with minimum samples equal to one: unique coordinates are connected when their haversine distance is within the radius, and union-find forms connected components. The default radius is 50 miles. A spatial grid reduces comparison work, with longitude handling adjusted for latitude and dateline wrapping. Radius must be positive and finite.

This permits chaining. City A can be near B, and B near C, so all three share a cluster even if A is far from C. Clusters are descriptive summaries only. They do not decide route eligibility and cannot admit a distant load merely through a chain of cities.

## 11.5 Directional lane eligibility

A load matches a lane only if its actual pickup is within the radius of that lane's start and its actual delivery is within the radius of that lane's end. Matching is directional; a reversed load needs the reversed definition. Eligible matches are sorted by the sum of those two endpoint offsets. This is geographic endpoint eligibility, not an actual road-distance comparison.

The batch runner considers all matched lane waypoint sequences, deduplicates identical via-coordinate/text sequences, resolves the lane waypoints and requests road routes between the actual load endpoints. Each returned road route goes through the same planRoute, access validation and fixed-fill optimiser as the ordinary planner. Per-load values override the base fuel rules. It chooses the cheapest successful plan among those tried paths.

Batch execution is sequential, reports progress, permits cancellation and retains completed load results. Unmatched or failed loads are separate statuses; there is no invented route or price. Export and review functions remain present. The load export's lane label currently uses the first eligible match, which may differ from the lane that produced the cheapest winning plan. Re-enabling the hidden workflow should include a regression review of that label and the old hidden waypoint-field paths.

Source: batch.js:1-151; route-library.json; workspace.js:1648-1848, 1953-2277.

# 12. Verification evidence and limits of the model

## 12.1 What was verified

The source build succeeded. The test runner reported 53 passed, zero failed, zero skipped and zero cancelled tests in approximately 3.27 seconds in the documentation verification copy. Repeated timing will vary. The current Hybrid update manifest matched all 22 protected/replaced file hashes that it references. No project source was edited.

Calculation tests cover one-gallon half-fill behavior, starting fuel above half target, half-mode infeasibility, zero-stop plans, starting below reserve, distant cheap but unreachable stations, inbound/outbound fuel, ending tolerance, strict rescue, same-mile alternatives, malformed/conflicting input and replay corruption. Geometry tests cover segment projection, revisited stations and California entry. Load tests cover alias parsing, clustering chains and endpoint eligibility.

The generated comparison uses a deterministic pseudo-random sequence, seven candidate stations per case and an independent enumeration of station subsets. Across 600 inputs it varies route distance, tank size, MPG, starting fuel, reserve, full/half target, stop limit, strict/buffer mode and ending tolerance. That is strong evidence for the small finite model. It is not exhaustive over every floating-point number, large production dataset or all California/rescue combinations; those have targeted tests instead.

## 12.2 What remains outside the guarantee

| Boundary               | Current behavior / implication                                                                                    |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| Entire road network    | Only provider-returned and user-selected routes are compared.                                                     |
| Purchase flexibility   | One global Full or Half target; no arbitrary gallons, budgets or per-stop target choice.                          |
| Operating cost         | Immediate fuel purchase spend only; no time/wage/toll/stop-cost objective or terminal fuel credit.                |
| Consumption            | Constant MPG; no grade, payload, traffic, weather, idling or driver-specific prediction.                          |
| Availability           | Loaded price/stop records only; no station-opening, diesel availability, parking or card-acceptance API.          |
| Corridor               | Geographic radius, followed by road-access pricing; no one-road-mile cap or same-side guarantee.                  |
| Access model           | Return to the same route anchor; no globally rerouted multi-stop path.                                            |
| HGV compliance         | Provider request fields, not an independent legal validation; payload details require live-provider confirmation. |
| Price entitlement      | No enforced client-to-fuel-master mapping in this snapshot.                                                       |
| Freshness              | Snapshot values and caches; no continuous re-optimisation or guaranteed 60-second data age.                       |
| Boundary entry         | Sampled/simplified California geometry can miss a crossing within one segment.                                    |
| Dispatch wording       | Editable message template can differ from Half Tank quantities.                                                   |
| Scale                  | O(KN squared) search plus potentially expensive road-access calls.                                                |
| Validation environment | Local build, pure functions, JSDOM and mocked provider tests; no live-network acceptance test.                    |

## 12.3 Safe extension boundaries

Changing the appearance, labels, map imagery or message layout should not change optimiser math. A new telematics provider can supply the same normalised fuel/location/MPG contract without changing the solver. A new road provider must supply compatible geometry, miles and directional access distances; it requires adapter and acceptance tests.

Adding arbitrary purchase quantities, stop fees, reserve relaxation, per-leg MPG, a road-distance eligibility cap, inventory valuation or route optimisation across station visits changes the mathematical model. Such changes need explicit requirements, new state/proof or a different optimiser, independent exhaustive comparisons where feasible, and regression checks of every displayed/exported quantity. They must not be introduced as a cosmetic update.

## 12.4 Diagnostic order for a surprising plan

- Check the actual selected client, visible fuel-source badge and price rows; these are separate selections.
- Check capacity, fuel gallons, MPG and any manually protected values on the form.
- Check route freshness, ordered waypoint resolution and the road profile/provider.
- Check whether a desired station is inside the geographic corridor and has confirmed directional road access.

- Check the mode's true hard reserve and any upper arrival/destination window.
- Compare purchase spend and destination fuel separately; a lower pump price can still require more total purchases.
- Check warnings, rescueUsed and whether all selected route alternatives completed.
- Replay the JSON arithmetic before changing any optimiser logic.

The formulas, worked examples and source index in this volume make that review reproducible. Part II provides the API diagnostics, persistence behavior, operational setup and file-level maintenance map needed to investigate the supplying data and interface.

Source: tests.mjs; verification run dated 13 September 2026; HYBRID-UPDATE-MANIFEST.json.

# A. Planning source index

Line numbers refer to the unchanged uploaded snapshot. The narrative explains the call relationships; this index locates each named calculation and routing helper. Internal anonymous callbacks are documented with their owning operation.

| File / symbol                    | Line | Role in source          |
|----------------------------------|------|-------------------------|
| optimizer.js / required          | 12   | Internal named function |
| optimizer.js / validateRules     | 19   | Exported function       |
| optimizer.js / arrivalAllowed    | 61   | Internal named function |
| optimizer.js / normaliseStations | 67   | Internal named function |
| optimizer.js / run               | 95   | Internal named function |
| optimizer.js / solveFuelPlan     | 219  | Exported function       |
| optimizer.js / verifyPlan        | 257  | Exported function       |
| geo.js / haversine               | 5    | Exported function       |
| geo.js / bearing                 | 24   | Internal named function |
| geo.js / move                    | 33   | Internal named function |
| geo.js / segmentProjection       | 54   | Exported function       |
| geo.js / prepareRoute            | 65   | Exported function       |
| geo.js / routeCandidates         | 84   | Exported function       |
| geo.js / pointAtMile             | 129  | Exported function       |
| geo.js / insidePolygon           | 144  | Exported function       |
| geo.js / californiaEntry         | 166  | Exported function       |
| geo.js / parseCoordinates        | 182  | Exported function       |
| batch.js / parseLoads            | 3    | Exported function       |
| batch.js / clusterCities         | 62   | Exported function       |
| batch.js / prepareLoads          | 122  | Exported function       |

| Integration point             | Source location          | Responsibility                                               |
|-------------------------------|--------------------------|--------------------------------------------------------------|
| Current rules / route options | workspace.js:319 / 339   | Read live form values and routing restrictions.              |
| Trip stamp / stale status     | workspace.js:349 / 360   | Separate route changes from calculation-only changes.        |
| Find routes                   | workspace.js:1099        | Resolve addresses, request geometry, select returned routes. |
| Road access enrichment        | workspace.js:1204        | Check every candidate, cache directed spur distances.        |
| Worker / route plan           | workspace.js:1282 / 1311 | Run solver and attach route/source metadata.                 |

| Integration point            | Source location          | Responsibility                                               |
|------------------------------|--------------------------|--------------------------------------------------------------|
| Selected route orchestration | workspace.js:1358        | Compare successful plans across selected alternatives.       |
| Fuel profile / results       | workspace.js:1416 / 1432 | Render the audit trail and final cards.                      |
| Export                       | workspace.js:1522        | Replay and download CSV/JSON.                                |
| Batch                        | workspace.js:1758        | Solve actual load endpoints through eligible lane waypoints. |
| Route points / HGV options   | server.js:765 / 785      | Validate coordinates and restrictions.                       |
| Directions / routes          | server.js:805 / 826      | ORS or OSRM geometry.                                        |
| Access / stop geometry       | server.js:888 / 966      | Directional distances and visual connectors.                 |